EE5203 L04 Lab Guide

Control LD2 through CMSIS registers - Basri Erdogan

Mission: Configure PA5 and blink the Nucleo-F401RE on-board LED with direct CMSIS register writes. Calculate every mask, inspect the changed registers, and explain why the physical result agrees with the C statements.

Prepare before class

- ☐ Bring the board, USB data cable, and working L03 project.
- ☐ Review pointers, `->`, hexadecimal, and bit masks.
- ☐ Complete the entry ticket:

```
int x = 5;
int *p = &x;
*p = 9;
```

1. What is stored in `p`? 2. What is the final value of `x`?
2. Which expression selects bit 5?

Hardware and register facts

Use the Nucleo-F401RE, VS Code, and a working L03 project copied as `EE5203_L04_<studentnumber>`. GPIO ownership is direct CMSIS register access; only `HAL_Delay` remains. Evidence comes from the build, SFR view, and LD2.

CMSIS expression	Required field
`RCC->AHB1ENR`	bit 0 enables GPIOA
`GPIOA->MODER`	bits 11:10 select PA5 mode
`GPIOA->BSRR`	bit 5 sets PA5; bit 21 resets PA5

Verify the field names in the STM32F401 reference manual. CMSIS provides the C names and addresses through the device header.

Warm-up (first 15 minutes)

Without running code, complete the values:

Purpose	Expression	Hex value
GPIOA clock bit	`1U << 0`	
PA5 mode field	`3U << 10`	
PA5 output pattern	`1U << 10`	
Set PA5	`1U << 5`	
Reset PA5	`1U << 21`	

Explain why PA5 begins at mode bit `5 × 2 = 10`.

Checkpoint A - Copy, build, and remove the old GPIO call

1. Copy and rename the L03 project; build with zero errors.
2. Keep generated initialization calls intact.
3. Remove the old `HAL_GPIO_TogglePin(...)` call; keep `HAL_Delay(...)`.
4. Confirm that `main.h` is included and the target is STM32F401RE.

Evidence: Show the project name, target device, and successful baseline build.

Checkpoint B - Decode one CMSIS register expression

```
GPIOA->MODER
```

Explain: `GPIOA` is a device-header pointer; `->` selects a member through that pointer; and `MODER` is a 32-bit register.

After generated initialization, execute this statement and inspect bits 11:10:

```
GPIOA->MODER &= ~(3U << 10);   // temporary input-mode baseline
```

Confirm `00`. This also establishes a known starting value for Checkpoint D.

Checkpoint C - Enable the GPIOA clock

Inside `USER CODE BEGIN 2`, before the generated `while (1)`, add:

```
RCC->AHB1ENR |= (1U << 0);
```

Predict bit 0, step over the statement, and inspect `RCC->AHB1ENR`. OR leaves all other clock bits unchanged.

Evidence: Record the observed register value and mark bit 0.

Checkpoint D - Configure PA5 as an output

```
GPIOA->MODER &= ~(3U << 10);
GPIOA->MODER |=  (1U << 10);
```

The first statement clears both PA5 mode bits; the second writes output pattern `01`. Break after each statement and record:

Moment	Predicted bits 11:10	Observed bits 11:10
After clear	`00`	
After set	`01`	

Checkpoint E - Turn LD2 on and off

Inside the existing generated `while (1)`:

```
GPIOA->BSRR = (1U << 5);
HAL_Delay(250);

GPIOA->BSRR = (1U << 21);
HAL_Delay(250);
```

Bits `0...15` set pins; bits `16...31` reset them. PA5 reset therefore uses bit `5 + 16 = 21`. Do not use a HAL GPIO write or toggle call.

Evidence: Demonstrate a stable blink and show both mask calculations.

Checkpoint F - Match debugger evidence to LD2

Break after the PA5 set write, inspect GPIOA in the SFR/peripheral view, and confirm LD2 is on. Continue to the reset write and confirm the opposite state. Do not step through `HAL_Delay`.

Record three claims: `MODER` bits 11:10 are `01`; the set-state observation agrees with LD2; and the reset-state observation agrees with LD2.

Checkpoint G - Individual controlled change

Predict first, make the assigned change, and collect new evidence.

Variant	Required change
A	Change both delays to `100` ms; explain why masks do not change.
B	Keep LD2 on by removing only the reset phase.
C	Replace literal `5` with `#define LED_PIN 5U` in both BSRR masks.

Acceptance checklist and oral check

- ☐ Build succeeds; no HAL GPIO write/toggle call remains.
- ☐ GPIOA clock bit 0 is set and PA5 mode bits are `01`.
- ☐ BSRR bit 5 sets LD2 and bit 21 resets it.
- ☐ Prediction, register evidence, and physical evidence agree.
- ☐ The assigned individual change is demonstrated.

Oral check: explain `GPIOA->MODER`, `3U << 10`, BSRR assignment, or why a successful build does not prove LED behavior.

Submit

Submit the relevant `main.c` user-code sections, completed tables, one register-view screenshot, one LD2 photograph/video, and three sentences explaining the controlled change.

Do not submit the entire workspace unless specifically requested.